



Query the database

Once you have [connected to the database](#), you can run queries using [Cypher](#) [?] and the method `Driver.execute_query()`.

TIP

`Driver.execute_query()` was introduced with the version 5.8 of the driver.

For queries with earlier versions, use [sessions and transactions](#).

Write to the database

To create a node representing a person named `Alice`, use the Cypher clause [CREATE](#):

Create a node representing a person named **Alice**

```
summary = driver.execute_query(
    "CREATE (:Person {name: $name})", ①
    name="Alice", ②
    database_"neo4j", ③
).summary
print("Created {nodes_created} nodes in {time} ms.".format(
    nodes_created=summary.counters.nodes_created,
    time=summary.result_available_after
))
```

- ① The Cypher query
- ② A map of *query parameters*
- ③ Which database the query should be run against

Read from the database

To retrieve information from the database, use the Cypher clause [MATCH](#):

Retrieve all **Person** nodes



AI search

```

records, summary, keys = driver.execute_query(
    "MATCH (p:Person) RETURN p.name AS name",
    database_="neo4j",
)

# Loop through results and do something with them
for record in records: ❶
    print(record.data()) # obtain record as dict

# Summary information ❷
print("The query `{query}` returned {records_count} records in {time} ms.".format(
    query=summary.query, records_count=len(records),
    time=summary.result_available_after
))

```

- ❶ records contains the result as an array of `Record` objects
- ❷ summary contains the summary of execution returned by the server

Update the database

To update a node's information in the database, use the Cypher clauses MATCH and SET:

Update node **Alice** to add an **age** property

```

records, summary, keys = driver.execute_query("""
    MATCH (p:Person {name: $name})
    SET p.age = $age
    """, name="Alice", age=42,
    database_="neo4j",
)
print(f"Query counters: {summary.counters}.")

```

To create a new relationship, linking it to two already existing node, use a combination of the Cypher clauses `MATCH` and `CREATE`:

Create a relationship :**KNOWS** between **Alice** and **Bob**

```

records, summary, keys = driver.execute_query("""
    MATCH (alice:Person {name: $name}) ❶
    MATCH (bob:Person {name: $friend}) ❷
    CREATE (alice)-[:KNOWS]->(bob) ❸
    """)

```

```

        """ , name="Alice", friend="Bob",
        database_="neo4j",
    )
    print(f"Query counters: {summary.counters}.")

```

- ① Retrieve the person node named `Alice` and bind it to a variable `alice`
- ② Retrieve the person node named `Bob` and bind it to a variable `bob`
- ③ Create a new `:KNOWS` relationship outgoing from the node bound to `alice` and attach to it the `Person` node named `Bob`

Delete from the database

To remove a node and any relationship attached to it, use the Cypher clause **DETACH DELETE** :

Remove the **Alice** node

```

records, summary, keys = driver.execute_query("""
    MATCH (p:Person {name: $name})
    DETACH DELETE p
    """ , name="Alice",
    database_="neo4j",
)
print(f"Query counters: {summary.counters}.")

```

Query parameters

Do not hardcode or concatenate parameters directly into queries. Instead, always use placeholders and specify the Cypher parameters, as shown in the previous examples. This is for:

1. **performance benefits:** Neo4j compiles and caches queries, but can only do so if the query structure is unchanged;
2. **security reasons:** see protecting against Cypher injection.

Query parameters can be passed either as several keyword arguments, or grouped together in a dictionary as value to the `parameters_` keyword argument. In case of mix, keyword-argument parameters take precedence over dictionary ones.

Pass query parameters as keyword arguments

```
driver.execute_query(
    "MERGE (:Person {name: $name})",
    name="Alice", age=42,
    database_="neo4j",
)
```

Pass query parameters in a dictionary

```
parameters = {
    "name": "Alice",
    "age": 42
}
driver.execute_query(
    "MERGE (:Person {name: $name})",
    parameters_=parameters,
    database_="neo4j",
)
```

None of your keyword query parameters may end with a single underscore. This is to avoid collisions with the [keyword configuration parameters](#). If you need to use such parameter names, pass them in the `parameters_` dictionary.

NOTE

There can be circumstances where your query structure prevents the usage of parameters in all its parts. For those rare use cases, see [Dynamic values in property keys, relationship types, and labels](#).

Error handling

Because `.execute_query()` can potentially raise a number of different exceptions, the best way to handle errors is to catch all exceptions in a single `try/except` block:

```
try:
    driver.execute_query(...)
except Exception as e:
    ... # handle exception
```

TID

The driver automatically retries to run a failed query, if the failure is deemed to be transient (for example due to temporary server unavailability). An exception will be raised if the operation keeps failing after a number of attempts.

Query configuration

You can supply further keyword arguments to alter the default behavior of `.execute_query()`. Configuration parameters are suffixed with `_`.

Database selection

It is recommended to **always specify the database explicitly** with the `database_` parameter, even on single-database instances. This allows the driver to work more efficiently, as it saves a network round-trip to the server to resolve the home database. If no database is given, the user's home database is used.

```
driver.execute_query(
    "MATCH (p:Person) RETURN p.name",
    database_="neo4j",
)
```

TIP

Specifying the database through the configuration method is preferred over the USE Cypher clause. If the server runs on a cluster, queries with `USE` require server-side routing to be enabled. Queries may also take longer to execute as they may not reach the right cluster member at the first attempt, and need to be routed to one containing the requested database.

Request routing

In a cluster environment, all queries are directed to the leader node by default. To improve performance on read queries, you can use the argument `routing_="r"` to route a query to the read nodes.

```
driver.execute_query(
    "MATCH (p:Person) RETURN p.name",
```

```
    routing_="r", # short for neo4j.RoutingControl.READ
    database_="neo4j",
)
```

NOTE

Although executing a *write* query in read mode likely results in a runtime error, **you should not rely on this for access control**. The difference between the two modes is that *read* transactions will be routed to any node of a cluster, whereas *write* ones are directed to primaries. In other words, there is no guarantee that a write query submitted in read mode will be rejected.

Run queries as a different user

Introduced in 5.14

You can execute a query through a different user with the parameter `auth_`. Switching user at the query level is cheaper than creating a new `Driver` object. The query is then run within the security context of the given user (i.e., home database, permissions, etc.).

Query-scoped authentication requires a server version ≥ 5.8 .

```
driver.execute_query(
    "MATCH (p:Person) RETURN p.name",
    auth_=("somebody_else", "their_password"),
    database_="neo4j",
)
```

The parameter `impersonated_user_` provides a similar functionality, and is available in driver/server versions ≥ 4.4 . The difference is that you don't need to know a user's password to impersonate them, but the user under which the `Driver` was created needs to have the appropriate permissions.

```
driver.execute_query(
    "MATCH (p:Person) RETURN p.name",
    impersonated_user_="somebody_else",
    database_="neo4j",
)
```

Transform query result

You can transform a query's result into a different data structure using the `result_transformer_` argument. The driver provides built-in methods to transform the result into a pandas dataframe or into a graph, but you can also craft your own transformer.

For more information, see [Manipulate query results](#).

A full example

```
from neo4j import GraphDatabase

URI = "<URI for Neo4j database>"
AUTH = ("<Username>", "<Password>")

people = [{"name": "Alice", "age": 42, "friends": ["Bob", "Peter", "Anna"]},
          {"name": "Bob", "age": 19},
          {"name": "Peter", "age": 50},
          {"name": "Anna", "age": 30}]

with GraphDatabase.driver(URI, auth=AUTH) as driver:
    try:
        # Create some nodes
        for person in people:
            View all \(39 more lines\)
```

For more information see [API documentation → Driver.execute_query\(\)](#).

[Prev](#)[Next](#)[◀ Connect to the database](#)[Manipulate query results ▶](#)

Contents

- Write to the database
- Read from the database
- Update the database
- Delete from the database
- Query parameters
- Error handling
- Query configuration

- Database selection
- Request routing
- Run queries as a different user
- Transform query result
- A full example

LEARN

-  [Sandbox](#)
-  [Neo4j Community Site](#)
-  [Neo4j Developer Blog](#)
-  [Neo4j Videos](#)
-  [GraphAcademy](#)
-  [Neo4j Labs](#)

SOCIAL

-  [Twitter](#)
-  [Meetups](#)
-  [Github](#)
-  [Stack Overflow](#)
- [Want to Speak?](#)

CONTACT US →

- US: 1-855-636-4532
- Sweden +46 171 480 113
- UK: +44 20 3868 3223
- France: +33 (0) 1 88 46 13 20

© 2025 Neo4j, Inc.
[Terms](#) | [Privacy](#) | [Sitemap](#)

Neo4j®, Neo Technology®, Cypher®, Neo4j® Bloom™ and Neo4j® Aura™ are registered trademarks of Neo4j, Inc. All other marks are owned by their respective companies.